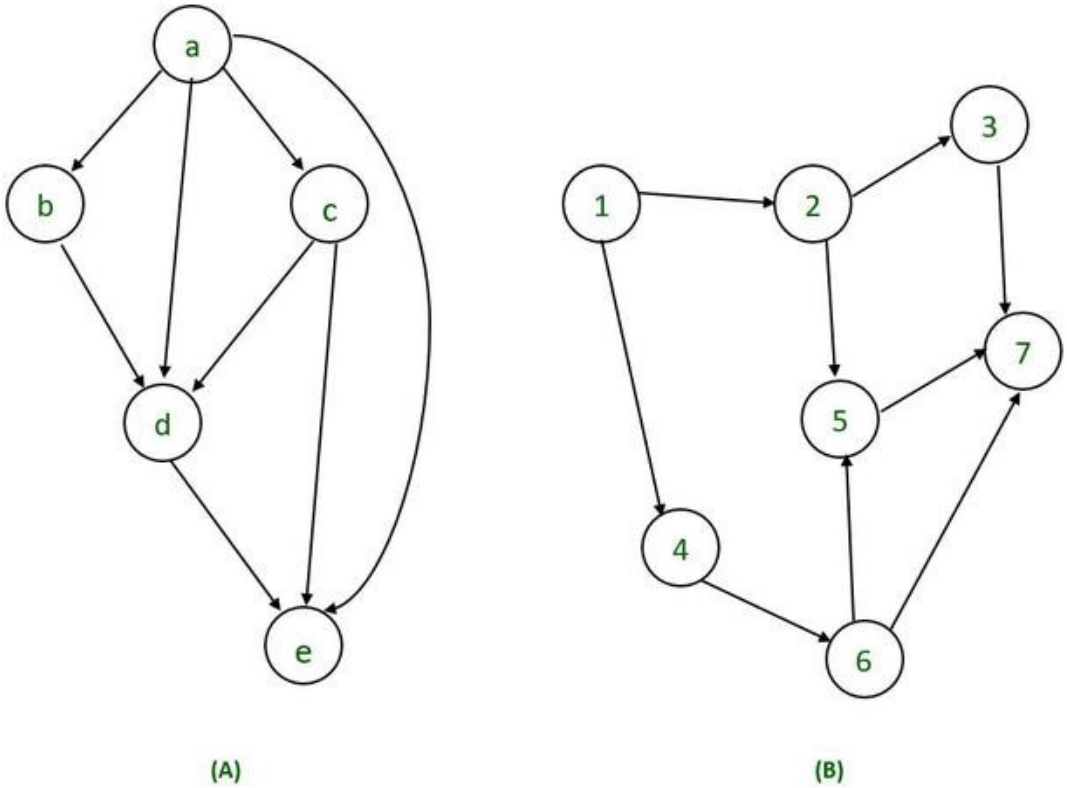
Definition of basic block control flow graphs

Directed Acyclic Graph :

The Directed Acyclic Graph (DAG) is used to represent the structure of basic blocks, to visualize the flow of values between basic blocks, and to provide optimization techniques in the basic block. To apply an optimization technique to a basic block, a DAG is a three-address code that is generated as the result of an intermediate code generation.

- Directed acyclic graphs are a type of data structure and they are used to apply transformations to basic blocks.
- The Directed Acyclic Graph (DAG) facilitates the transformation of basic blocks.
- DAG is an efficient method for identifying common sub-expressions.
- It demonstrates how the statement's computed value is used in subsequent statements.

Examples of directed acyclic graph :



Directed Acyclic Graph Characteristics :

A Directed Acyclic Graph for Basic Block is a directed acyclic graph with the following labels on nodes.

- 4 The graph's leaves each have a unique identifier, which can be variable names or constants.
- 5 The interior nodes of the graph are labelled with an operator symbol.
- 6 In addition, nodes are given a string of identifiers to use as labels for storing the computed value.
- 7 Directed Acyclic Graphs have their own definitions for transitive closure and transitive reduction.
- 8 Directed Acyclic Graphs have topological orderings defined.

Algorithm for construction of Directed Acyclic Graph :

There are three possible scenarios for building a DAG on three address codes:

Case 1 – x = y op z

Case 2 – x = op y

Case 3 – x = y

Directed Acyclic Graph for the above cases can be built as follows :

Step 1 –

If the y operand is not defined, then create a node (y).

If the z operand is not defined, create a node for case(1) as node(z)

Step 2 –

Create node(OP) for case(1), with node(z) as its right child and node(OP) as its left child (y).

For the case (2), see if there is a node operator (OP) with one child node (y).

Node n will be node(y) in case (3).

Step 3 –

Remove x from the list of node identifiers. Step 2: Add x to the list of attached identifiers for node n.

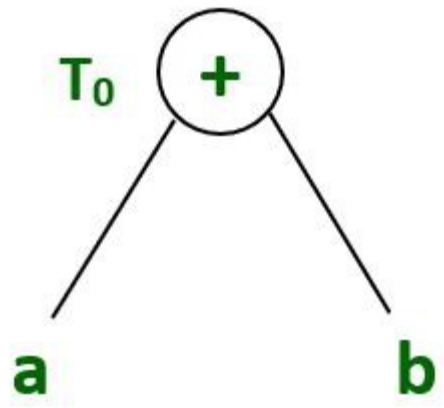
Example :

T0 = a + b —Expression 1

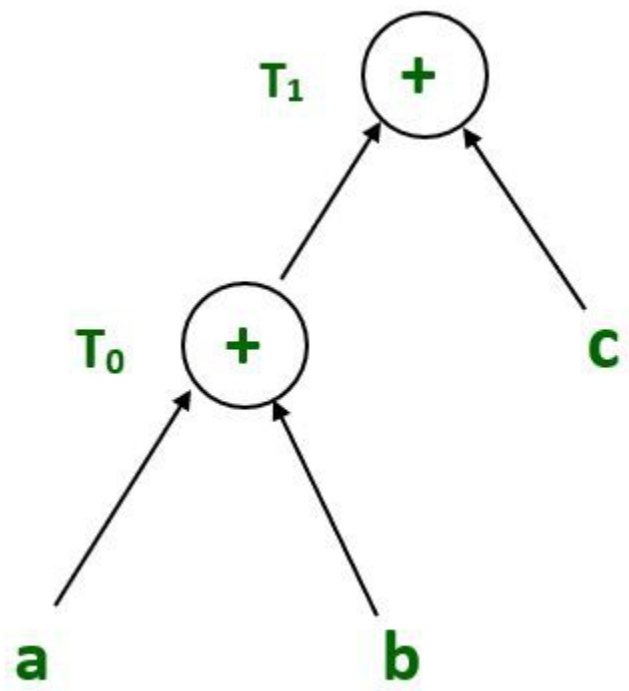
T1 = T0 + c —-Expression 2

d = T0 + T1 ——Expression 3

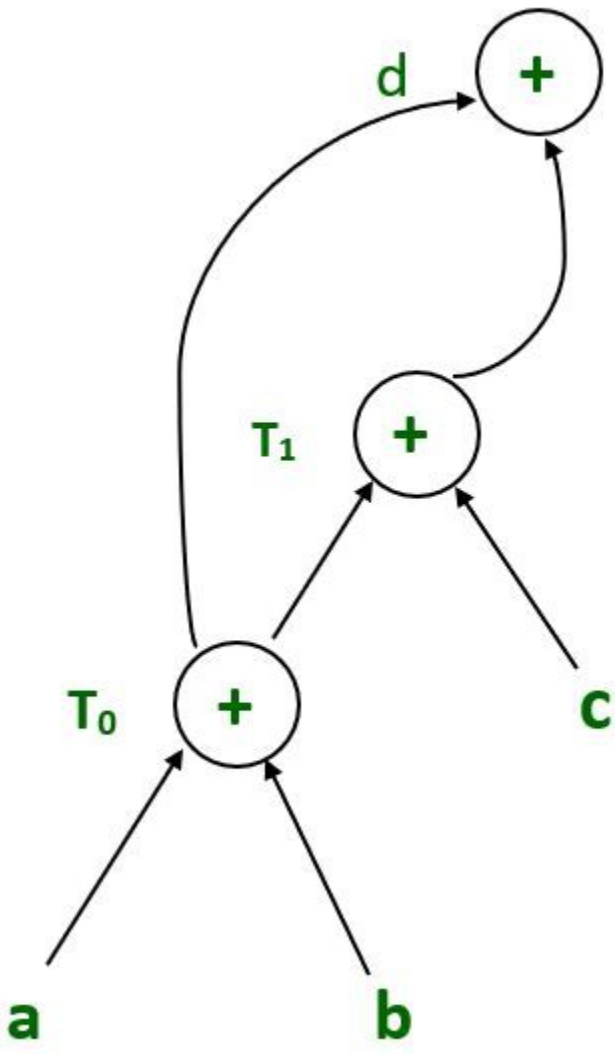
Expression 1 : T0 = a + b



Expression 2: T1 = T0 + c



Expression 3 : d = T0 + T1

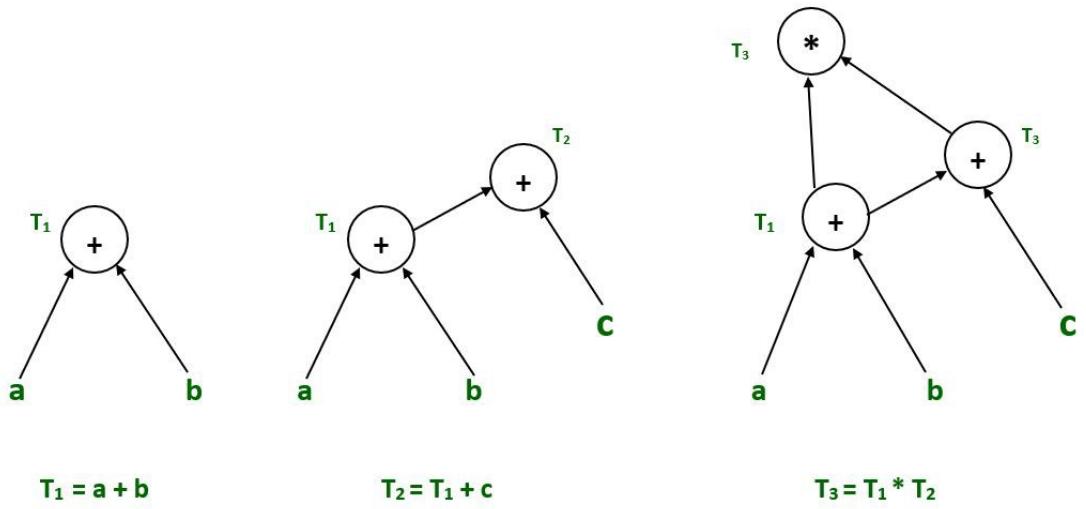


Example :

T1 = a + b

T2 = T1 + c

T3 = T1 x T2



Application of Directed Acyclic Graph:

Directed acyclic graph determines the subexpressions that are commonly used.

Directed acyclic graph determines the names used within the block as well as the names computed outside the block.

Determines which statements in the block may have their computed value outside the block.

Code can be represented by a Directed acyclic graph that describes the inputs and outputs of each of the arithmetic operations performed within the code; this representation allows the compiler to perform common subexpression elimination efficiently.

Several programming languages describe value systems that are linked together by a directed acyclic graph. When one value changes, its successors are recalculated; each value in the DAG is evaluated as a function of its predecessors.

Merits/ Advantages of using DAG:

More flexible and communicative.

No transaction fee

Higher scalability

Everyone is responsible for both issuing and validating transactions.

Network can easily scale

The Principal Sources of Optimization

1. Causes of Redundancy There are many redundant operations in a typical program. Sometimes the redundancy is available at the source level. ...

2. A Running Example: Quicksort ...

3. Semantics-Preserving Transformations ...

4. Global Common Subexpressions ...

5. Copy Propagation ...

Loop Optimization is the process of increasing execution speed and reducing the overheads associated with loops. It plays an important role in improving cache performance and making effective use of parallel processing capabilities. Most execution time of a scientific program is spent on loops.

Loop Optimization is a machine independent optimization.

Decreasing the number of instructions in an inner loop improves the running time of a program even if the amount of code outside that loop is increased.

Loop Optimization Techniques:

1, Frequency Reduction (Code Motion):

In frequency reduction, the amount of code in loop is decreased. A statement or expression, which can be moved outside the loop body without affecting the semantics of the program, is moved outside the loop.

Example:

Initial code:

```
while(i<100)
{
 a = Sin(x)/Cos(x) + i;
```

```
 i++;
}
```

Optimized code:

```
t = Sin(x)/Cos(x);
while(i<100)
{
 a = t + i;
 i++;
}
```

2.**Loop Unrolling**:

Loop unrolling is a loop transformation technique that helps to optimize the execution time of a program. We basically remove or reduce iterations. Loop unrolling increases the program's speed by eliminating loop control instruction and loop test instructions.

Example:

Initial code:

```
for (int i=0; i<5; i++)
    printf("Pankaj\n");
```

Optimized code:

```
printf("Pankaj\n");
```

3.Loop Jamming:

Loop jamming is the combining the two or more loops in a single loop. It reduces the time taken to compile the many number of loops.

Example:

Initial Code:

```
for(int i=0; i<5; i++)
   a = i + 5;
for(int i=0; i<5; i++)
   b = i + 10;
```

Optimized code:

```
for(int i=0; i<5; i++)
{
 a = i + 5;
 b = i + 10;
}
```

Loop Invariant Condition with Examples

Definition:

A loop invariant is a condition [among program variables] that is necessarily true immediately before and immediately after each iteration of a loop. (Note that this says nothing about its truth or falsity part way through an iteration.)

A loop invariant is some predicate (condition) that holds for every iteration of the loop.

For example, let's look at a simple for loop that looks like this:

```
int j = 9;
for(int i=0; i<10; i++)
j--;
```

In this example it is true (for every iteration) that i + j == 9.

A weaker invariant that is also true is that i >= 0 && i <= 10.

One may get confused between the loop invariant, and the loop conditional

(the condition which controls termination of the loop).

The loop invariant must be true:

before the loop starts

before each iteration of the loop

after the loop terminates

(although it can temporarily be false during the body of the loop).

On the other hand the loop conditional must be false after the loop terminates, otherwise, the loop would never terminate.

Usage:Loop invariants capture key facts that explain why code works. This means that if you write code in which the loop invariant is not obvious, you should add a comment that gives the loop invariant. This helps other programmers understand the code, and helps keep them from accidentally breaking the invariant with future changes.

A loop Invariant can help in the design of iterative algorithms when considered an assertion that expresses important relationships among the variables that must be true at the start of every iteration and when the loop terminates. If this holds, the computation is on the road to effectiveness. If false, then the algorithm has failed.

Loop invariants are used to reason about the correctness of computer programs. Intuition or trial and error can be used to write easy algorithms however when the complexity of the problem increases, it is better to use formal methods such as loop invariants.

- Loop invariants can be used to prove the correctness of an algorithm, debug an existing algorithm without even tracing the code or develop an algorithm directly from specification.A good loop invariant should satisfy three properties:
- Initialization: The loop invariant must be true before the first execution of the loop.
- Maintenance: If the invariant is true before an iteration of the loop, it

should be true also after the iteration.

- Termination: When the loop is terminated the invariant should tell us something useful, something that helps us understand the algorithm.

Loop Invariant Condition:

Loop invariant condition is a condition about the relationship between the variables of our program which is definitely true immediately before and immediately after each iteration of the loop.

For example: Consider an array A{7, 5, 3, 10, 2, 6} with 6 elements and we have to find maximum element max in the array.

```
max = -INF (minus infinite)

for (i = 0 to n-1)

  if (A[i] > max)

      max = A[i]
```

In the above example after the 3rd iteration of the loop max value is 7, which holds true for the first 3 elements of array A. Here, the loop invariant condition is that max is always maximum among the first i elements of array A.

Loop Invariant condition of various algorithms:

- Selection Sort:

In selection sort algorithm we find the minimum element from the unsorted part and put it at the beginning.

```
min_idx = 0


for (i = 0; i < n-1; i++)

{

   min_idx = i;

   for (j = i+1 to n-1)
```

```
    if (arr[j] < arr[min_idx])
      min_idx = j;


  swap(&arr[min_idx], &arr[i]);
}
```

In the above pseudo code there are two loop invariant condition:

In the outer loop, array is sorted for first i elements.

In the inner loop, min is always the minimum value in A[i to j].

- 14 Insertion Sort: in insertion sort, loop invariant condition is that the subarray A[0 to i-1] is always sorted.

```
for (i = 1 to n-1)
{
  key = arr[i];
  j = i-1;
  while (j >= 0 and arr[j] > key)
{
    arr[j+1] = arr[j];
    j = j-1;
  }
  arr[j+1] = key;
}
```

Peephole Optimization

Peephole optimization is a type of code Optimization performed on a small part of the code. It is performed on a very small set of instructions in a segment of code.

The small set of instructions or small part of code on which peephole

optimization is performed is known as peephole or window.

It basically works on the theory of replacement in which a part of code is replaced by shorter and faster code without a change in output. The peephole is machine-dependent optimization.

Objectives of Peephole Optimization: The objective of peephole optimization is as follows**:**

- ·15 To improve performance
- ·16 To reduce memory footprint
- ·17 To reduce code size

Peephole Optimization Techniques

A. Redundant load and store elimination: In this technique, redundancy is eliminated.

Initial code:

y = x + 5;

i = y;

z = i;

w = z * 3;

Optimized code:

y = x + 5;

i = y;

w = y * 3;

B. Constant folding: The code that can be simplified by the user itself, is simplified.

Initial code:

x = 2 * 3;

Optimized code:x = 6;

C. Strength Reduction: The operators that consume higher execution time

are replaced by the operators consuming less execution time.

Initial code:

y = x * 2;

Optimized code:

y = x + x; or y = x << 1;

Initial code:

y = x / 2;

Optimized code:

y = x >> 1;

D. Null sequences: Useless operations are deleted.

E. Combine operations: Several operations are replaced by a single equivalent operation.

Issues in the design of a code generator

Code generator converts the intermediate representation of source code into a form that can be readily executed by the machine. A code generator is expected to generate the correct code. Designing of the code generator should be done in such a way so that it can be easily implemented, tested, and maintained.

The following issue arises during the code generation phase:

Input to code generator – The input to the code generator is the intermediate code generated by the front end, along with information in the symbol table that determines the run-time addresses of the data objects denoted by the names in the intermediate representation. Intermediate codes may be represented mostly

in quadruples, triples, indirect triples, Postfix notation, syntax trees, DAGs, etc. The code generation phase just proceeds on an assumption that the input is free from all syntactic and state semantic errors, the necessary type checking has taken place and the type-conversion operators have been inserted wherever necessary.

·18 Target program: The target program is the output of the code

generator. The output may be absolute machine language, relocatable machine language, or assembly language.

·19 Absolute machine language as output has advantages that it can be placed in a fixed memory location and can be immediately executed.

·20 Relocatable machine language as an output allows subprograms and subroutines to be compiled separately. Relocatable object modules can be linked together and loaded by a linking loader. But there is added expense of linking and loading.

·21 Assembly language as output makes the code generation easier. We can generate symbolic instructions and use the macro-facilities of assemblers in generating code. And we need an additional assembly step after code generation

·22 Memory Management – Mapping the names in the source program to the addresses of data objects is done by the front end and the code generator. A name in the three address statements refers to the symbol table entry for the name. Then from the symbol table entry, a relative address can be determined for the name.

·23 Instruction selection – Selecting the best instructions will improve the efficiency of the program. It includes the instructions that should be complete and uniform. Instruction speeds and machine idioms also play a major role when efficiency is considered. But if we do not care about the efficiency of the target program then instruction selection is straightforward. For example, the respective three-

address statements would be translated into the latter code sequence as shown below:

P:=Q+R

S:=P+T

MOV Q, R0

ADD R, R0

MOV R0, P

MOV P, R0

ADD T, R0

MOV R0, S

Here the fourth statement is redundant as the value of the P is loaded again in that statement that just has been stored in the previous statement. It leads to an inefficient code sequence. A given intermediate representation can be translated into many code sequences, with significant cost differences between the different implementations. A prior knowledge of instruction cost is needed in order to design good sequences, but accurate cost information is difficult to predict.

·24 Register allocation issues – Use of registers make the computations faster in comparison to that of memory, so efficient utilization of registers is important. The use of registers are subdivided into two subproblems:

·25 During Register allocation – we select only those set of variables that will reside in the registers at each point in the program.

·26 During a subsequent Register assignment phase, the specific register is picked to access the variable.

·27 Evaluation order – The code generator decides the order in which the instruction will be executed. The order of computations affects the efficiency of the target code. Among many computational orders, some will require only fewer registers to hold the intermediate results. However, picking the best order in the general case is a difficult NP-complete problem.

·28 Approaches to code generation issues: Code generator must always generate the correct code. It is essential because of the number of special cases that a code generator might face. Some of the design goals of code generator are:Correct

Easily maintainable

Testable

Efficient

Target code generation is the final Phase of Compiler.

Input : Optimized Intermediate Representation.

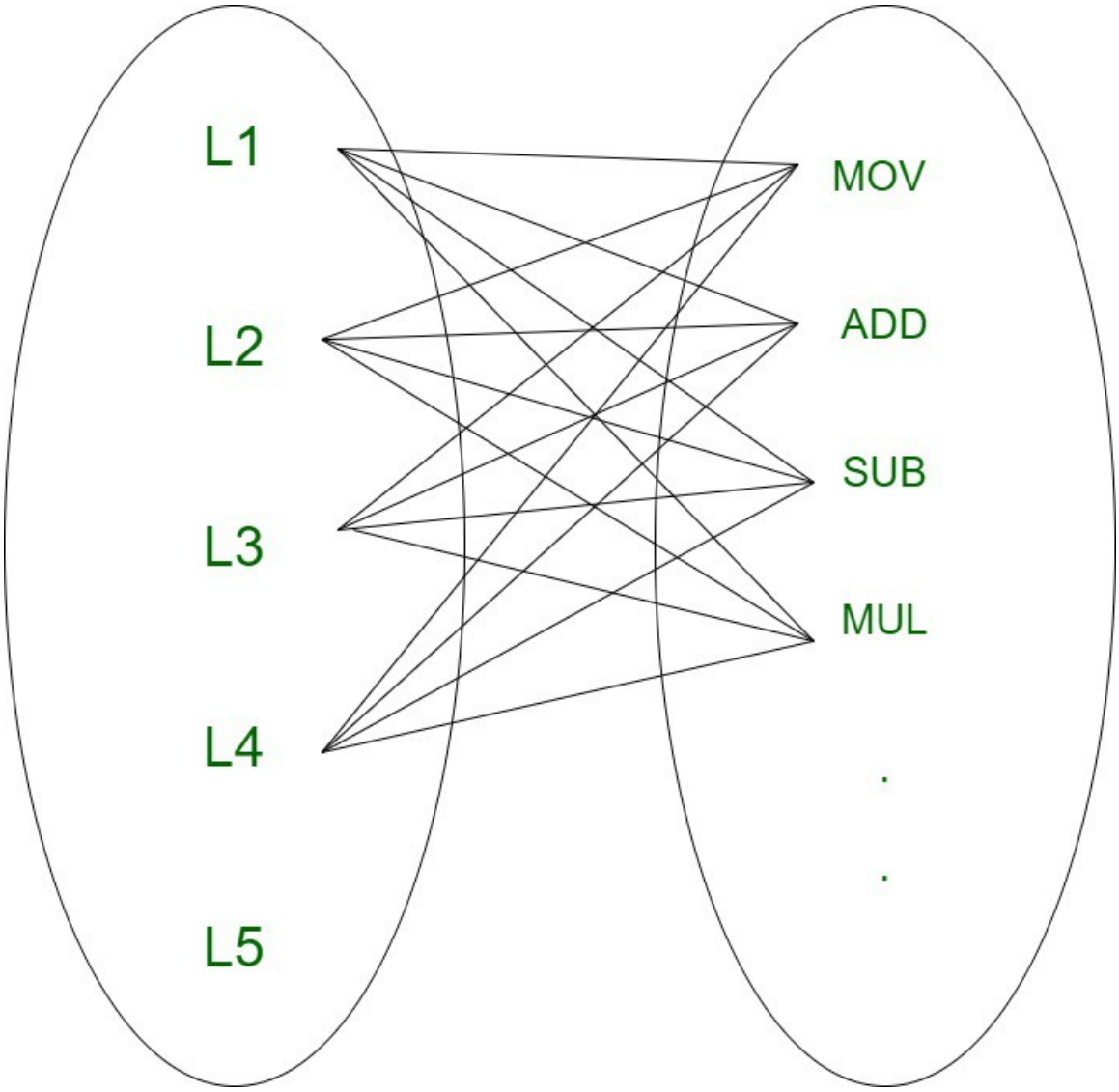
Output : Target Code.

Task Performed : Register allocation methods and optimization, assembly level code.

Method : Three popular strategies for register allocation and optimization.

Implementation : Algorithms.

Target code generation deals with assembly language to convert optimized code into machine understandable format. Target code can be machine readable code or assembly code. Each line in optimized code may map to one or more lines in machine (or) assembly code, hence there is a 1:N mapping associated with them



Optimized code :

Example 1 :

L1: a = b + c * d

optimization :

t0 = c * d

a = b + t0

Example 2 :

L2: e = f - g / d

optimization :

t0 = g / d

e = f - t0

Register Allocation :

Register allocation is the process of assigning program variables to registers and reducing the number of swaps in and out of the registers. Movement of variables across memory is time consuming and this is the main reason why registers are used as they available within the memory and they are the fastest accessible storage location.

Example 1:

R1<--- a

R2<--- b

R3<--- c

R4<--- d

MOV R3, c

MOV R4, d

MUL R3, R4

MOV R2, b

ADD R2, R3

MOV R1, R2

MOV a, R1

Example 2:

R1<--- e

R2<--- f

R3<--- g

R4<--- h

MOV R3, g

MOV R4, h

DIV R3, R4

MOV R2, f

SUB R2, R3

MOV R1, R2

MOV e, R1

Advantages :

Fast accessible storage

Allows computations to be performed on them

Deterministic as it incurs no miss

Reduce memory traffic

Reduces overall computation time

Disadvantages :

Registers are generally available in small amount (up to few hundred Kb)

Register sizes are fixed and it varies from one processor to another

Registers are complicated

Need to save and restore changes during context switch and procedure calls

Code

Generation from DAG.

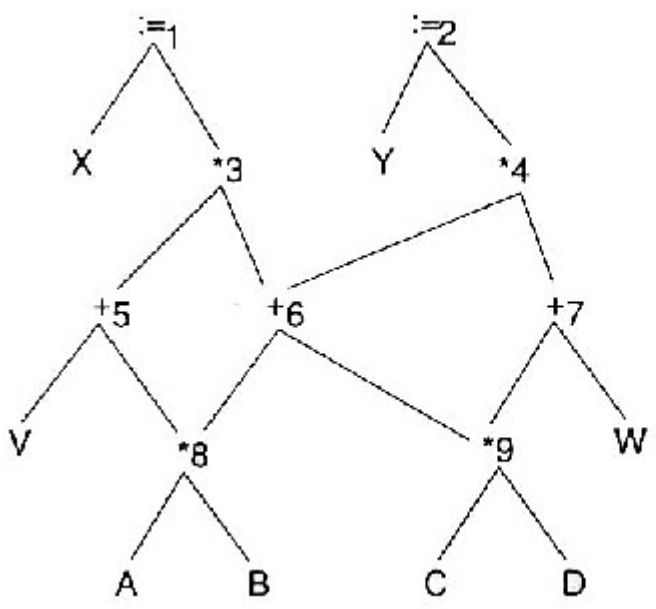
The optimal code generation algorithm is an NP-complete problem. Directed acyclic graphs allow a code Generation heuristic:

Put out code for each node immediately after code for its children has been emitted as far as possible because then the results are more apt to still be available, say in a register.

To prepare the list of DAG nodes to compute (that is, the list for which code is to be emitted), start at the root of the right-most subtree. Put this node on the list, L, and continue by adding a left-most node to the list after all its parents are already on the list. Then generate code for the nodes in L by starting at the end of L and proceeding to the beginning.

Example 1

Generating code from a DAG



2 4 7 1 3 5 6 9 8

Thus, we would put out code for node 8 first, then node 9, etc.

Another way to put out good code from DAG's is to break the DAG up into

trees and to use a code generation algorithm which is optimal on trees. (Of course, it takes time to break up the DAG.)